ACREDITACIÓN 5 AÑOS

Instituto Profesional Virginio Gómez
Universidad de Concepción

Arquitectura Avanzada en C#: Interfaces y Polimorfismo Transversal

Caso Aplicado: Sistema de Auditoría Agroindustrial — Fundo IPVG

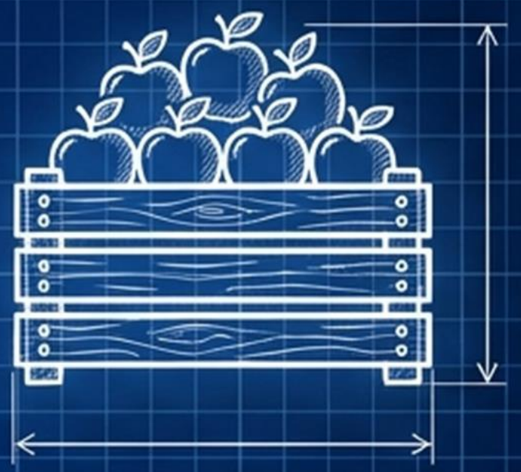
Docente | Ingeniería de Ejecución en Computación e Informática | Plataforma .NET

El límite de la herencia tradicional

- En un Fundo Agroindustrial, necesitamos auditar financieramente objetos de naturalezas completamente distintas.
- Un Tractor y un LoteManzana no comparten un ancestro lógico común (no superan la prueba es-un).
- Forzar una clase base ObjetoAuditable rompe la cohesión y el encapsulamiento. ¿Cómo estandarizamos su comportamiento sin forzar herencia?



Tractor (Maquinaria)



LoteManzana (Producto Agrícola)

ObjetoAuditable

⚠ ERROR DE DISEÑO

La solución: Contratos de Comportamiento

Contrato estricto

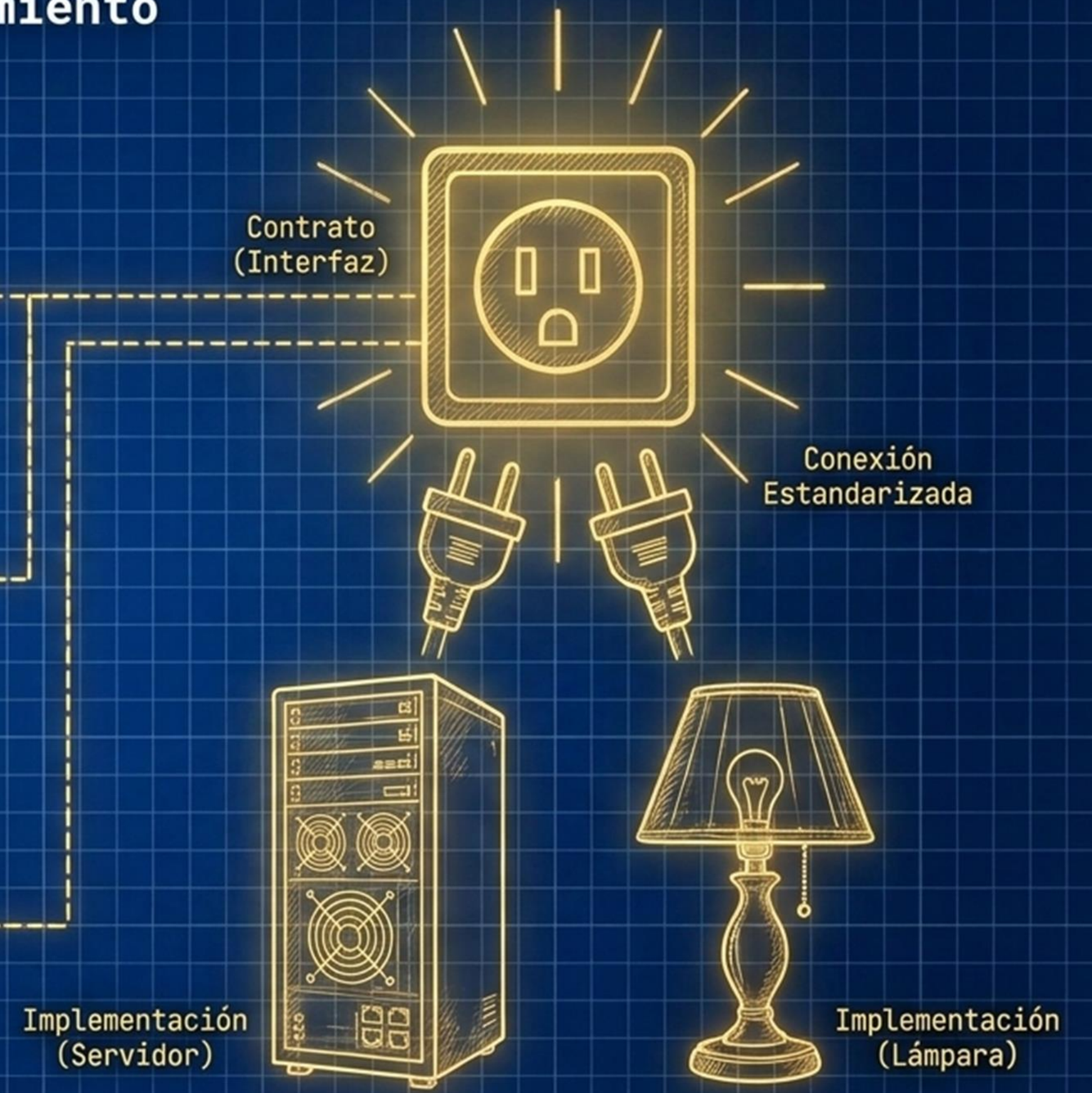
Una **interface** en C# define un contrato estricto.

Capacidad (No identidad)

No define qué es el objeto, sino qué es capaz de hacer.

Garantía de implementación

Garantiza que cualquier clase que firme el contrato implementará obligatoriamente los métodos y propiedades definidos, independientemente de su posición en la jerarquía de clases.



Matriz Estructural: Clase Abstracta vs. Interfaz

Dimensión	Clase Abstracta (abstract class)	Interfaz (interface)
Propósito	Define la identidad base (Es un...)	Define capacidades transversales (Puede...)
Estado	Puede contener campos (variables) y constructores.	No contiene estado (campos ni constructores).
Herencia en C#	Simple (Una clase hereda solo de una clase base).	Múltiple (Una clase puede implementar múltiples interfaces).
Implementación	Puede tener métodos con código (virtuales).	En .NET moderno admite implementaciones por defecto, pero su rol principal es la firma pura.

Definiendo nuestro primer contrato



```
namespace SistemaAuditoria
{
    /// <summary>
    /// Contrato para cualquier elemento del fundo que tenga valor financiero.
    /// </summary>
    public interface IValorable
    {
        // Propiedad obligatoria
        string CodigoAuditoria { get; }

        // Método obligatorio (Firma pura, sin cuerpo)
        decimal CalcularValorMercado();
    }
}
```

Conectando las piezas al contrato

```
public class Tractor : IValorable
{
    public string CodigoAuditoria { get; set; }
    public decimal Depreciacion { get; set; }

    public decimal CalcularValorMercado()
    {
        return 15000000m - Depreciacion;
    }
}
```

CONTRATO FIRMADO

```
public class LoteManzana : IValorable
{
    public string CodigoAuditoria { get; set; }
    public decimal Kilos { get; set; }

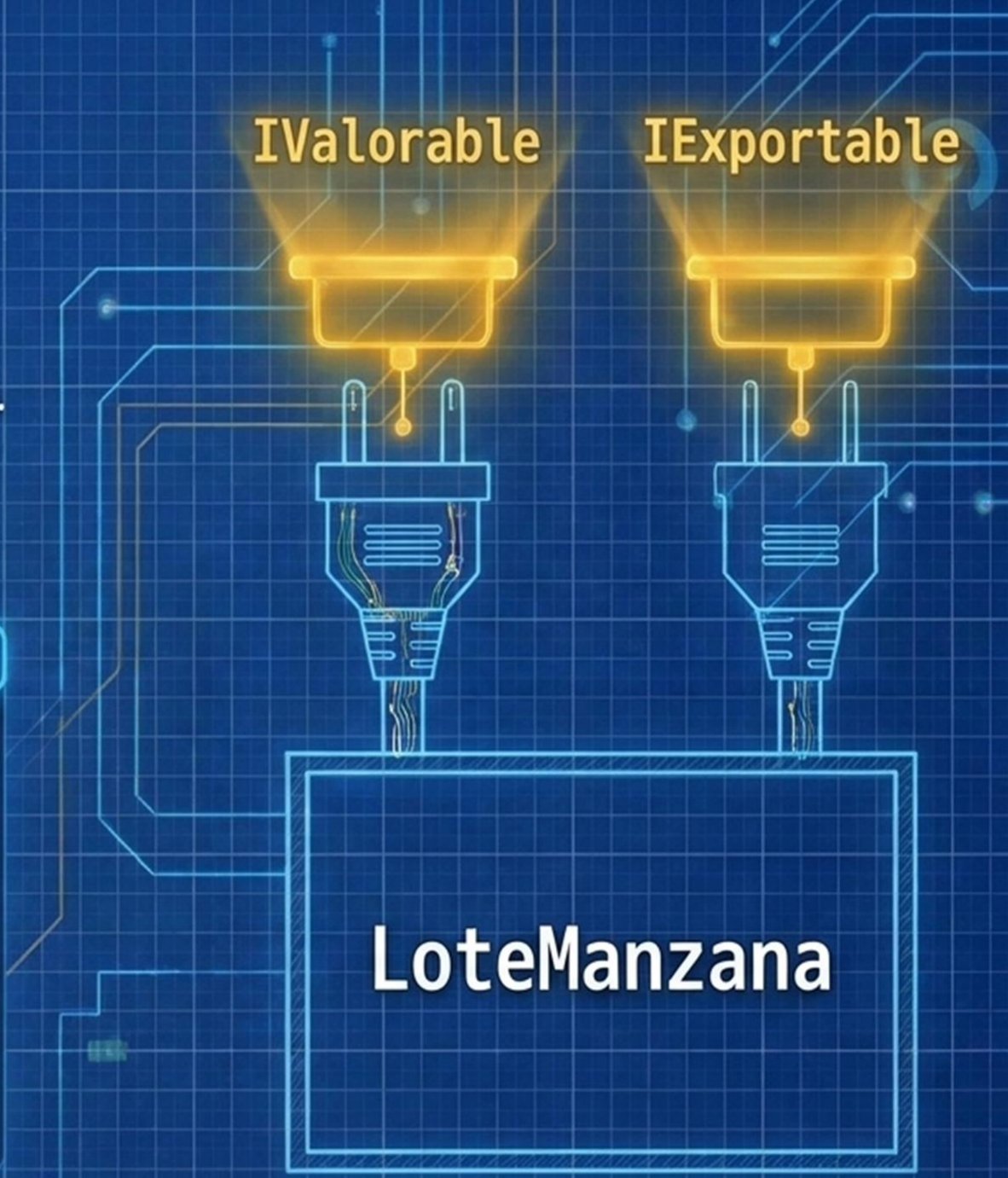
    public decimal CalcularValorMercado()
    {
        return Kilos * 850m; // Precio por kilo
    }
}
```

Rompiendo el límite: Implementación Múltiple

A diferencia de las clases, una entidad en C# puede firmar múltiples contratos simultáneamente.

Esto permite una arquitectura altamente modular sin jerarquías frágiles.

```
1  public interface IExportable
2  {
3      bool CumpleNormativaSAG();
4  }
5
6  // Implementación Múltiple separada por comas
7  public class LoteManzana : IValorable, IExportable
8  {
9      // ... implementación de ambas interfaces ...
10 }
```



Arquitectura del Fundo IPVG

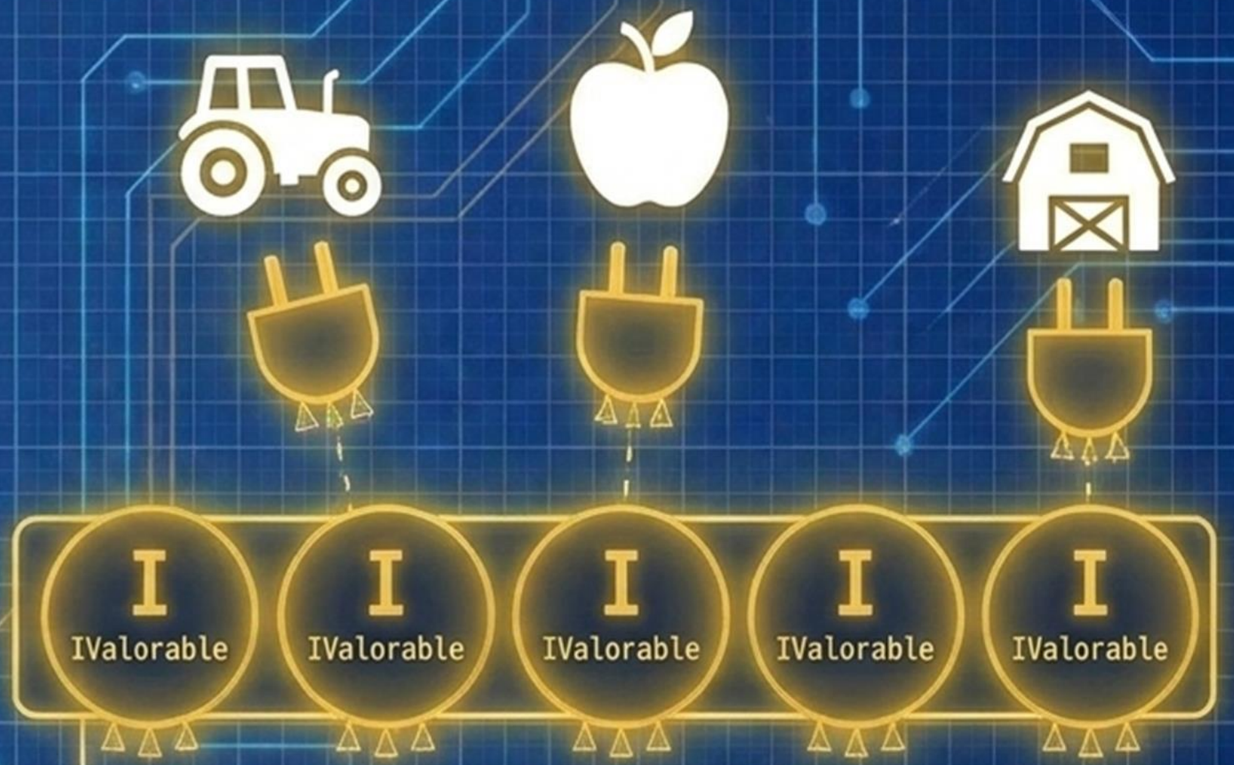


Mapeo visual de responsabilidades. Observe cómo el polimorfismo transversal atraviesa entidades de distintas familias de objetos.

Colecciones Genéricas y Tipos Abstractos

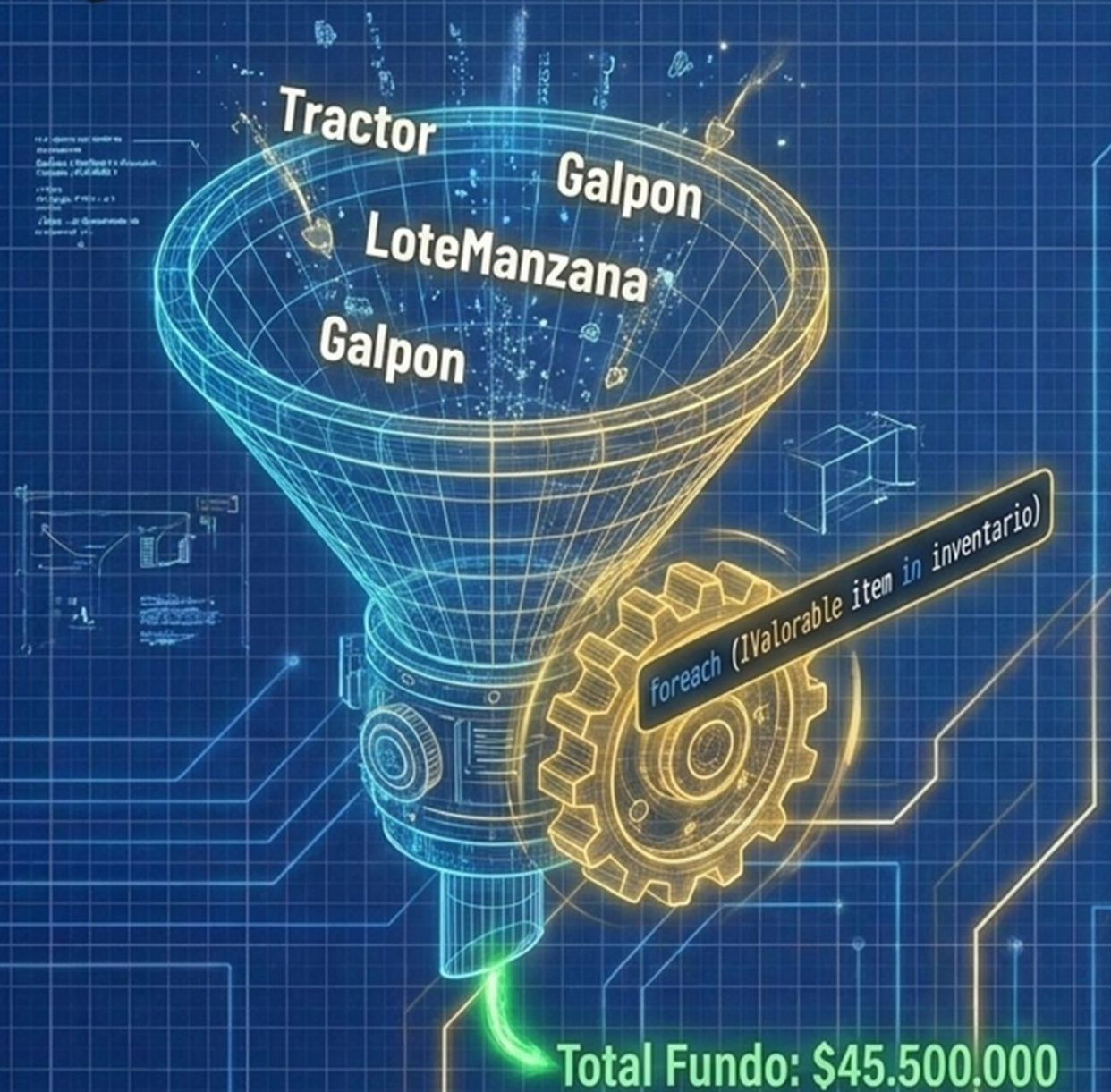
La plataforma .NET provee System.Collections.Generic.List<T>.

Al tipar la lista con una interfaz (**List<IValorable>**), creamos una colección que acepta cualquier clase que haya firmado el contrato.




```
1  List<IValorable> inventario = new List<IValorable>();
2
3  inventario.Add(new Tractor());
4  inventario.Add(new LoteManzana()); // ¡Colección polimórfica!
```

Flujo Funcional de Auditoría




```
decimal patrimonioTotal = 0;

foreach (IValorable activo in inventario)
{
    // Polimorfismo puro: C# decide en tiempo de ejecución
    // qué método CalcularValorMercado() ejecutar.
    patrimonioTotal += activo.CalcularValorMercado();
}
```

Taller Práctico: Sistema de Auditoría Fundo IPVG



Contexto: El Fundo requiere un software de consola para auditar su patrimonio antes de una fiscalización.

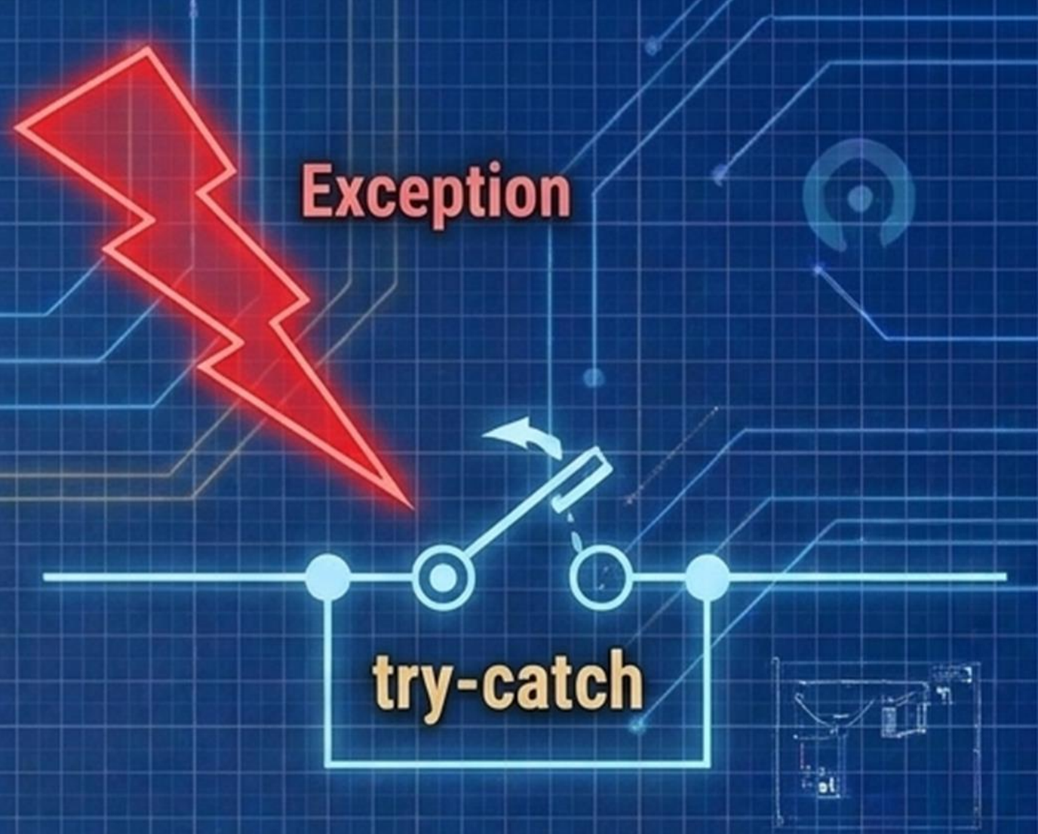
Requerimientos:

1. Implementar interfaz IValorable.
2. Crear clases Tractor, LoteManzana e InsumoQuimico.
3. Agrupar al menos 5 instancias en una List<IValorable>.
4. Construir un menú interactivo (do-while / switch) para imprimir el reporte total.

Robustez del Sistema: Gestión de Excepciones

¿Qué sucede si un usuario ingresa un valor negativo para la depreciación del tractor?

Debemos proteger la ejecución utilizando bloques **try-catch** para evitar caídas catastróficas en tiempo de ejecución.




```
try
{
    decimal valor = activo.CalcularValorMercado();
    if (valor < 0)
        throw new InvalidOperationException("El valor de mercado no puede ser negativo.");
}
catch (Exception ex)
{
    Console.WriteLine($"Error en auditoría: {ex.Message}");
}
```